```
#zip function
#takes iterables, aggregate in tuple
#zip(iterables)

seismic_attributes = ['Amplitude', 'Frequency', 'Phase', 'Coherence',
'Instantaneous Amplitude']
interpretations = ['Hydrocarbon Indicator', 'Reservoir Quality',
'Structural Features', 'Fault Detection', 'Signal Strength']

attributes_pairs = zip(seismic_attributes, interpretations)
attributes_pairs_list  = list(attributes_pairs)
print(attributes_pairs_list)
```

```
[('Amplitude', 'Hydrocarbon Indicator'), ('Frequency', 'Reservoir
Quality'), ('Phase', 'Structural Features'), ('Coherence', 'Fault
Detection'), ('Instantaneous Amplitude', 'Signal Strength')]
```

```
bulk_densities = [2.4, 2.35, 2.3, 2.25, 2.2]
matrix_densities = [2.65, 2.7, 2.75, 2.6, 2.55]
rho_fluid = 1

porosities = [ round((rho_matrix - rho_bulk)/(rho_matrix - rho_fluid),
4)
               for rho_matrix, rho_bulk in zip(bulk_densities,
matrix_densities) ]


print(porosities)
```

```
[-0.1786, -0.2593, -0.3462, -0.28, -0.2917]
```

```
attributes_pairs_list
```

```
[('Amplitude', 'Hydrocarbon Indicator'),
 ('Frequency', 'Reservoir Quality'),
 ('Phase', 'Structural Features'),
 ('Coherence', 'Fault Detection'),
 ('Instantaneous Amplitude', 'Signal Strength')]
```

```
#zip(*Zipped_list)

attributes, interp = zip(*attributes_pairs_list)

print(attributes)
print(interp)
```

```
('Amplitude', 'Frequency', 'Phase', 'Coherence', 'Instantaneous
Amplitude')
('Hydrocarbon Indicator', 'Reservoir Quality', 'Structural Features',
'Fault Detection', 'Signal Strength')
```

DocStrings

```python
#Multiplication of two numbers function
def multipl(a,b):

    '''
    This is a function
    Multiplication of two numbers
    should be integers

    '''
    print(a*b)

multipl(52, 85)
```

```
4420
```

```python
print(multipl.__doc__)
```

```
    This is a function
    Multiplication of two numbers
    should be integers



```

```python
print(list.__doc__)
```

```
Built-in mutable sequence.

If no argument is given, the constructor creates a new empty list.
The argument must be an iterable if specified.
```

```python
print(zip.__doc__)
```

```
zip(*iterables, strict=False) --> Yield tuples until an input is
exhausted.

   >>> list(zip('abcdefg', range(3), range(4)))
   [('a', 0, 0), ('b', 1, 1), ('c', 2, 2)]

The zip object yields n-length tuples, where n is the number of
iterables
passed as positional arguments to zip().  The i-th element in every
tuple
comes from the i-th iterable argument to zip().  This continues until
the
shortest argument is exhausted.

If strict is true and one of the arguments is exhausted before the
others,
raise a ValueError.
```

```python
help(zip)
```

```
Help on class zip in module builtins:

class zip(object)
 |  zip(*iterables, strict=False) --> Yield tuples until an input is
exhausted.
 |
 |      >>> list(zip('abcdefg', range(3), range(4)))
 |      [('a', 0, 0), ('b', 1, 1), ('c', 2, 2)]
 |
 |  The zip object yields n-length tuples, where n is the number of
iterables
 |  passed as positional arguments to zip().  The i-th element in
every tuple
 |  comes from the i-th iterable argument to zip().  This continues
until the
 |  shortest argument is exhausted.
 |
 |  If strict is true and one of the arguments is exhausted before the
others,
 |  raise a ValueError.
 |
 |  Methods defined here:
 |
 |  __getattribute__(self, name, /)
 |      Return getattr(self, name).
 |
 |  __iter__(self, /)
 |      Implement iter(self).
 |
 |  __next__(self, /)
 |      Implement next(self).
 |
 |  __reduce__(...)
 |      Return state information for pickling.
 |
 |  __setstate__(...)
 |      Set state information for unpickling.
 |
 |
----------------------------------------------------------------------
 |  Static methods defined here:
 |
 |  __new__(*args, **kwargs)
 |      Create and return a new object.  See help(type) for accurate
signature.
```

```
help(list)
```

```
Help on class list in module builtins:

class list(object)
 |  list(iterable=(), /)
 |
 |  Built-in mutable sequence.
 |
 |  If no argument is given, the constructor creates a new empty list.
 |  The argument must be an iterable if specified.
 |
 |  Methods defined here:
 |
 |  __add__(self, value, /)
 |      Return self+value.
 |
 |  __contains__(self, key, /)
 |      Return bool(key in self).
 |
 |  __delitem__(self, key, /)
 |      Delete self[key].
 |
 |  __eq__(self, value, /)
 |      Return self==value.
 |
 |  __ge__(self, value, /)
 |      Return self>=value.
 |
 |  __getattribute__(self, name, /)
 |      Return getattr(self, name).
 |
 |  __getitem__(self, index, /)
 |      Return self[index].
 |
 |  __gt__(self, value, /)
 |      Return self>value.
 |
 |  __iadd__(self, value, /)
 |      Implement self+=value.
 |
 |  __imul__(self, value, /)
 |      Implement self*=value.
 |
 |  __init__(self, /, *args, **kwargs)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  __iter__(self, /)
 |      Implement iter(self).
 |
 |  __le__(self, value, /)
```

```
 |      Return self<=value.
 |
 |  __len__(self, /)
 |      Return len(self).
 |
 |  __lt__(self, value, /)
 |      Return self<value.
 |
 |  __mul__(self, value, /)
 |      Return self*value.
 |
 |  __ne__(self, value, /)
 |      Return self!=value.
 |
 |  __repr__(self, /)
 |      Return repr(self).
 |
 |  __reversed__(self, /)
 |      Return a reverse iterator over the list.
 |
 |  __rmul__(self, value, /)
 |      Return value*self.
 |
 |  __setitem__(self, key, value, /)
 |      Set self[key] to value.
 |
 |  __sizeof__(self, /)
 |      Return the size of the list in memory, in bytes.
 |
 |  append(self, object, /)
 |      Append object to the end of the list.
 |
 |  clear(self, /)
 |      Remove all items from list.
 |
 |  copy(self, /)
 |      Return a shallow copy of the list.
 |
 |  count(self, value, /)
 |      Return number of occurrences of value.
 |
 |  extend(self, iterable, /)
 |      Extend list by appending elements from the iterable.
 |
 |  index(self, value, start=0, stop=9223372036854775807, /)
 |      Return first index of value.
 |
 |      Raises ValueError if the value is not present.
 |
 |  insert(self, index, object, /)
```

```
 |      Insert object before index.
 |
 |  pop(self, index=-1, /)
 |      Remove and return item at index (default last).
 |
 |      Raises IndexError if list is empty or index is out of range.
 |
 |  remove(self, value, /)
 |      Remove first occurrence of value.
 |
 |      Raises ValueError if the value is not present.
 |
 |  reverse(self, /)
 |      Reverse *IN PLACE*.
 |
 |  sort(self, /, *, key=None, reverse=False)
 |      Sort the list in ascending order and return None.
 |
 |      The sort is in-place (i.e. the list itself is modified) and
stable (i.e. the
 |      order of two equal elements is maintained).
 |
 |      If a key function is given, apply it once to each list item
and sort them,
 |      ascending or descending, according to their function values.
 |
 |      The reverse flag can be set to sort in descending order.
 |
 |
----------------------------------------------------------------------
 |  Class methods defined here:
 |
 |  __class_getitem__(...)
 |      See PEP 585
 |
 |
----------------------------------------------------------------------
 |  Static methods defined here:
 |
 |  __new__(*args, **kwargs)
 |      Create and return a new object.  See help(type) for accurate
signature.
 |
 |
----------------------------------------------------------------------
 |  Data and other attributes defined here:
 |
 |  __hash__ = None
```

Object oriented Programming (OOP)

```
help(list)

Help on class list in module builtins:

class list(object)
 |  list(iterable=(), /)
 |
 |  Built-in mutable sequence.
 |
 |  If no argument is given, the constructor creates a new empty list.
 |  The argument must be an iterable if specified.
 |
 |  Methods defined here:
 |
 |  __add__(self, value, /)
 |      Return self+value.
 |
 |  __contains__(self, key, /)
 |      Return bool(key in self).
 |
 |  __delitem__(self, key, /)
 |      Delete self[key].
 |
 |  __eq__(self, value, /)
 |      Return self==value.
 |
 |  __ge__(self, value, /)
 |      Return self>=value.
 |
 |  __getattribute__(self, name, /)
 |      Return getattr(self, name).
 |
 |  __getitem__(self, index, /)
 |      Return self[index].
 |
 |  __gt__(self, value, /)
 |      Return self>value.
 |
 |  __iadd__(self, value, /)
 |      Implement self+=value.
 |
 |  __imul__(self, value, /)
 |      Implement self*=value.
 |
 |  __init__(self, /, *args, **kwargs)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  __iter__(self, /)
```

```
 |      Implement iter(self).
 |
 |  __le__(self, value, /)
 |      Return self<=value.
 |
 |  __len__(self, /)
 |      Return len(self).
 |
 |  __lt__(self, value, /)
 |      Return self<value.
 |
 |  __mul__(self, value, /)
 |      Return self*value.
 |
 |  __ne__(self, value, /)
 |      Return self!=value.
 |
 |  __repr__(self, /)
 |      Return repr(self).
 |
 |  __reversed__(self, /)
 |      Return a reverse iterator over the list.
 |
 |  __rmul__(self, value, /)
 |      Return value*self.
 |
 |  __setitem__(self, key, value, /)
 |      Set self[key] to value.
 |
 |  __sizeof__(self, /)
 |      Return the size of the list in memory, in bytes.
 |
 |  append(self, object, /)
 |      Append object to the end of the list.
 |
 |  clear(self, /)
 |      Remove all items from list.
 |
 |  copy(self, /)
 |      Return a shallow copy of the list.
 |
 |  count(self, value, /)
 |      Return number of occurrences of value.
 |
 |  extend(self, iterable, /)
 |      Extend list by appending elements from the iterable.
 |
 |  index(self, value, start=0, stop=9223372036854775807, /)
 |      Return first index of value.
```

```
 |
 |      Raises ValueError if the value is not present.
 |
 |  insert(self, index, object, /)
 |      Insert object before index.
 |
 |  pop(self, index=-1, /)
 |      Remove and return item at index (default last).
 |
 |      Raises IndexError if list is empty or index is out of range.
 |
 |  remove(self, value, /)
 |      Remove first occurrence of value.
 |
 |      Raises ValueError if the value is not present.
 |
 |  reverse(self, /)
 |      Reverse *IN PLACE*.
 |
 |  sort(self, /, *, key=None, reverse=False)
 |      Sort the list in ascending order and return None.
 |
 |      The sort is in-place (i.e. the list itself is modified) and
stable (i.e. the
 |      order of two equal elements is maintained).
 |
 |      If a key function is given, apply it once to each list item
and sort them,
 |      ascending or descending, according to their function values.
 |
 |      The reverse flag can be set to sort in descending order.
 |
 |
----------------------------------------------------------------------
 |  Class methods defined here:
 |
 |  __class_getitem__(...)
 |      See PEP 585
 |
 |
----------------------------------------------------------------------
 |  Static methods defined here:
 |
 |  __new__(*args, **kwargs)
 |      Create and return a new object.  See help(type) for accurate
signature.
 |
 |
----------------------------------------------------------------------
```

```
 |  Data and other attributes defined here:
 |
 |  __hash__ = None
```

```
#syntax to create Class

#class Class_Name:              (Firts ltter should be capital)
    #variables   (Attributes)
    #Functions   (Methods)

#Syntax to create object

#object_name = Class_Name()

porosities = list()   #object

porosities.append(0.2)    #methods
print(porosities)
```

```
[0.2]
```

```
porosities.pop()
```

```
0.2
```

```
print(porosities)
```

```
[]
```

```
porosities = [ 0.2, 0.5, 0.6 , 0.4]
Sw = [ 0.3, 0.1, 0.12, 0.15]

#create class

class Well:
    name = ""        #default value
    depth = 0
    resrervoir = ""

#create objects

Well_1 = Well()
Well_1.name = "Well-A"
Well_1.depth = 3000
Well_1.resrervoir = "Sandstone"

Well_2 = Well()

Well_2.name = "Well-B"
Well_2.depth = 4500
Well_2.resrervoir = "Carbonate"
```

```
Well_1.name
```

```
'Well-A'
```

```
Well_2.depth
```

```
4500
```

```
Well_2.resrervoir
```

```
'Carbonate'
```

```
Well_1.resrervoir
```

```
'Sandstone'
```

```
class Wells:
    def __init__(self,name, depth, reservoir):   #__init__ function
        self.name = name
        self.depth = depth
        self.reservoir = reservoir
        print(f"Well name is : {self.name} and depth is {self.depth}
and reservoir is {self.reservoir}")

Well_4 = Wells("Well-a", 3000, "Sand")
Well_4 = Wells("Well-B", 4500, "Carbonate")
```

```
Well name is : Well-a and depth is 3000 and reservoir is Sand
Well name is : Well-B and depth is 4500 and reservoir is Carbonate
```

```
class Wells_methods:
    def __init__(self,name, depth, reservoir):   #__init__ function
        self.name = name
        self.depth = depth
        self.reservoir = reservoir
        print(f"Well name is : {self.name} and depth is {self.depth}
and reservoir is {self.reservoir}")

    def display_info(self):
        print(f"Resrvoir name is : {self.name}")
        print(f"depth is {self.depth}")
        print(f"reservoir type is {self.reservoir}")

Well_5 = Wells_methods("Well-a", 3000, "Sand")
Well_6 = Wells_methods("Well-B", 4500, "Carbonate")
```

```
Well name is : Well-a and depth is 3000 and reservoir is Sand
Well name is : Well-B and depth is 4500 and reservoir is Carbonate
```

```
Well_5.name
```

```
'Well-a'
```

```
Well_6.depth
```

```
4500
```

```
Well_5.display_info()
```

```
Resrvoir name is : Well-a
depth is 3000
reservoir type is Sand
```

```
Well_6.display_info()
```

```
Resrvoir name is : Well-B
depth is 4500
reservoir type is Carbonate
```

```
Well_2.display_info()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[47], line 1
----> 1 Well_2.display_info()

AttributeError: 'Well' object has no attribute 'display_info'
```

```
class Reservoir:
    formation_volume_factor = 1.3  # Common formation factor for all reservoirs

    def __init__(self, name, area, thickness, porosity, oil_saturation):
        print(f"{name} Reservoir object created!")
        self.name = name
        self.area = area  # m²
        self.thickness = thickness  # m
        self.porosity = porosity  # fraction
        self.oil_saturation = oil_saturation  # fraction

    def calc_OOIP(self):
        # OOIP calculation (in cubic meters)
        OOIP = ((self.area * self.thickness * self.porosity * self.oil_saturation)/self.formation_volume_factor)
        #print(f"OOIP for {self.name} reservoir is {OOIP} m3")
        return(f"OOIP for {self.name} reservoir is {OOIP:.3f} m3")

Reservoir_1 = Reservoir("Nukhul", 2000, 25, 0.18, 0.85)
```

```
Nukhul Reservoir object created!
```

```
Reservoir_2 = Reservoir("Nubia", 3500, 20, 0.2, 0.9)
```

```
Nubia Reservoir object created!
```

```
Reservoir_1.calc_OOIP()
```

```
'OOIP for Nukhul reservoir is 5884.615 m3'
```

```
Reservoir_2.calc_OOIP()
```

```
'OOIP for Nubia reservoir is 9692.308 m3'
```

```
#1 _ Abstraction: Simplifying, Hiding internal complexity


#list.append()

#2_Encapsulation  :
# 3 _ Name  Mangling

class SeismicSurvey:
    def __init__(self, project_name, _survey_area, __budget):
        self.project_name = project_name  # Public attribute
        self._survey_area = _survey_area  # Protected attribute
        self.__budget = __budget  # Private attribute

    def display_project(self):
        """Public method to display project details."""
        print(f"Project Name: {self.project_name}")

    def _display_area(self):
        """Protected method to display the survey area (for internal
use)."""
        print(f"Survey Area: {self._survey_area} sq km")

    def __display_budget(self):
        """Private method to display the project budget (not
accessible externally)."""
        print(f"Project Budget: ${self.__budget}")

    def access_budget(self):
        """Public method to access the private budget."""
        self.__display_budget()

Survey_1 = SeismicSurvey("North Field", 2000, 2000000)

Survey_1.project_name
```

```
'North Field'
```

```
Survey_1.display_project()
```

```
Project Name: North Field
```

```
Survey_1._survey_area
```

```
2000
```

```
Survey_1._display_area()
```

```
Survey Area: 2000 sq km
```

```
Survey_1.__budget
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[70], line 1
----> 1 Survey_1.__budget

AttributeError: 'SeismicSurvey' object has no attribute '__budget'
```

```
Survey_1.__display_budget()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[64], line 1
----> 1 Survey_1.__display_budget()

AttributeError: 'SeismicSurvey' object has no attribute '__display_budget'
```

```
Survey_1.access_budget()
```

```
Project Budget: $2000000
```

```
print(Survey_1.__dict__)
```

```
{'project_name': 'North Field', '_survey_area': 2000, '_SeismicSurvey__budget': 2000000}
```

```
Survey_1._SeismicSurvey__budget
```

```
2000000
```

```
Survey_1._SeismicSurvey__budget = 5000000
```

```
print(Survey_1.__dict__)
```

```
{'project_name': 'North Field', '_survey_area': 2000, '_SeismicSurvey__budget': 5000000}
```

```
#Inheritance

class SeismicData:
    def __init__(self, survey_name, depth, wave_velocity):
```

```
        self.survey = survey_name
        self.depth = depth
        self.velocity = wave_velocity

    def describe_survey(self):
        return f"Survey: {self.survey}, Depth: {self.depth}m, Wave
Velocity: {self.velocity} m/s"

Seismic_info = SeismicData("North Field", 3000, 2500)

Seismic_info.depth

3000

Seismic_info.describe_survey()

'Survey: North Field, Depth: 3000m, Wave Velocity: 2500 m/s'

class Attributes:
    def __init__(self,attribute_name, attribute_value):

        self.attribute_name = attribute_name
        self.attribute_value  = attribute_value

    def describe_attribute(self):
        return f"{self.attribute_name} has a value of
{self.attribute_value} "

attribute = Attributes("amplitude", 65)

attribute.describe_attribute()

'amplitude has a value of 65 '

attribute.describe_survey()

---------------------------------------------------------------------------
-----
AttributeError                            Traceback (most recent call
last)
Cell In[82], line 1
----> 1 attribute.describe_survey()

AttributeError: 'Attributes' object has no attribute 'describe_survey'

class SeismicAttribute:
    def __init__(self, survey_name, depth, wave_velocity,
attribute_name, attribute_value):
        self.survey = survey_name
        self.depth = depth
        self.velocity = wave_velocity
        self.attribute_name = attribute_name
```

```
        self.attribute_value  = attribute_value

    def describe_survey(self):
        return f"Survey: {self.survey}, Depth: {self.depth}m, Wave
Velocity: {self.velocity} m/s"

    def describe_attribute(self):
        return f"{self.attribute_name} has a value of
{self.attribute_value} for {self.survey} survey."

seismic_info_2 = SeismicAttribute("North Field", 3000, 2500,
"amplitude", 75)

seismic_info_2.describe_survey()
```

```
'Survey: North Field, Depth: 3000m, Wave Velocity: 2500 m/s'
```

```
class SeismicAttribute(SeismicData):
    def __init__(self, survey_name, depth, wave_velocity,
attribute_name, attribute_value):
        super().__init__(survey_name, depth, wave_velocity)
        self.attribute_name = attribute_name
        self.attribute_value  = attribute_value
    def describe_attribute(self):
        return f"{self.attribute_name} has a value of
{self.attribute_name} for {self.survey} survey."

Seismic_info_3 = SeismicAttribute("North Field", 3000, 2500,
"amplitude", 75)

Seismic_info_3.describe_survey()
```

```
'Survey: North Field, Depth: 3000m, Wave Velocity: 2500 m/s'
```

```
class AVO_attributes(SeismicAttribute):
    def __init__(self, survey_name, depth, wave_velocity,
attribute_name, value, gradient, intercept):
        super().__init__(survey_name, depth, wave_velocity,
attribute_name, value)
        self.gradient = gradient
        self.intercept = intercept

    def avo_result(self):
        return f"AVO Analysis: Gradient = {self.gradient}, Intercept =
{self.intercept}"

Seismic_AVO = AVO_attributes("North Field", 2500, 3500, "Amplitude",
0.75, -0.2, 0.5)

Seismic_AVO.describe_survey()
```

```
'Survey: North Field, Depth: 2500m, Wave Velocity: 3500 m/s'
```

```
Seismic_AVO.describe_attribute()

'Amplitude has a value of Amplitude for North Field survey.'

Seismic_AVO.avo_result()

'AVO Analysis: Gradient = -0.2, Intercept = 0.5'

#Method Resolution Order (MRO)


help(AVO_attributes)

Help on class AVO_attributes in module __main__:

class AVO_attributes(SeismicAttribute)
 |  AVO_attributes(survey_name, depth, wave_velocity, attribute_name,
value, gradient, intercept)
 |
 |  Method resolution order:
 |      AVO_attributes
 |      SeismicAttribute
 |      SeismicData
 |      builtins.object
 |
 |  Methods defined here:
 |
 |  __init__(self, survey_name, depth, wave_velocity, attribute_name,
value, gradient, intercept)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  avo_result(self)
 |
 |
----------------------------------------------------------------------
 |  Methods inherited from SeismicAttribute:
 |
 |  describe_attribute(self)
 |
 |
----------------------------------------------------------------------
 |  Methods inherited from SeismicData:
 |
 |  describe_survey(self)
 |
 |
----------------------------------------------------------------------
 |  Data descriptors inherited from SeismicData:
 |
 |  __dict__
 |      dictionary for instance variables
```

```
 |
 |  __weakref__
 |      list of weak references to the object
```

```
#Polymorphism :


# Parent Class: SeismicData
class SeismicData:
    def __init__(self, survey_name, depth, wave_velocity):
        self.survey = survey_name
        self.depth = depth
        self.velocity = wave_velocity

    def describe_survey(self):
        return f"Survey: {self.survey}, Depth: {self.depth}m, Wave
Velocity: {self.velocity} m/s"

# Child Class: SeismicAttribute (Extending SeismicData)
class SeismicAttribute(SeismicData):
    def __init__(self, survey_name, depth, wave_velocity,
attribute_name, value):
        super().__init__(survey_name, depth, wave_velocity)
        self.attribute = attribute_name
        self.value = value

    # Polymorphism - Overriding describe_survey() to include attribute
details
    def describe_survey(self):
        return (f"Survey: {self.survey}, Depth: {self.depth}m, Wave
Velocity: {self.velocity} m/s, "
                f"Attribute: {self.attribute}, Value: {self.value}")

# Grandchild Class: AVOAnalysis (Extending SeismicAttribute)
class AVOAnalysis(SeismicAttribute):
    def __init__(self, survey_name, depth, wave_velocity,
attribute_name, value, gradient, intercept):
        super().__init__(survey_name, depth, wave_velocity,
attribute_name, value)
        self.gradient = gradient
        self.intercept = intercept

    # Further overriding describe_survey() to include AVO parameters
    def describe_survey(self):
        return (f"AVO Analysis for {self.survey}: Depth =
{self.depth}m, Velocity = {self.velocity} m/s, "
                f"Attribute = {self.attribute}, Value = {self.value},
Gradient = {self.gradient}, "
                f"Intercept = {self.intercept}")
```

```python
    def avo_result(self):
        return f"AVO Analysis: Gradient = {self.gradient}, Intercept = {self.intercept}"



# Creating instances of each class
basic_survey = SeismicData("North Basin Survey", 2500, 3500)
attribute_survey = SeismicAttribute("North Basin Survey", 2500, 3500, "Acoustic Impedance", 12000)
avo_analysis = AVOAnalysis("North Basin Survey", 2500, 3500, "Amplitude", 0.85, -0.1, 0.3)

basic_survey.describe_survey()
```

```
'Survey: North Basin Survey, Depth: 2500m, Wave Velocity: 3500 m/s'
```

```python
attribute_survey.describe_survey()
```

```
'Survey: North Basin Survey, Depth: 2500m, Wave Velocity: 3500 m/s, Attribute: Acoustic Impedance, Value: 12000'
```

```python
avo_analysis.describe_survey()
```

```
'AVO Analysis for North Basin Survey: Depth = 2500m, Velocity = 3500 m/s, Attribute = Amplitude, Value = 0.85, Gradient = -0.1, Intercept = 0.3'
```